SKILL LAB MILESTONE – 1
Entertainment Booking System
Name: Kripa K

A1. Identify the issues

The given schema has many problems because all data is stored in a single table.

Updating issue – if we want to update something like email, we need to change it in all rows. If we miss one place, data becomes inconsistent.

Insertion issue – we cannot insert new data independently. For example, adding a new theatre or movie is not possible without booking data.

Deletion issue – deleting one record may remove other important data also since everything is stored together.

2NF issue – user_name depends on user and not on user_id, so there is partial dependency.

Multiple values – seat numbers can be stored like A1, A2, A3 in one column which violates 1NF.

3NF issue – theatre_location depends on theatre_name and not on primary key, so it creates transitive dependency.

Redundancy – user email, theatre details and other data are repeated again and again in every booking row.

A2. Schema normalization

To remove these problems, the schema is divided into multiple tables.

```sql
CREATE DATABASE entertainment;

CREATE TABLE user(
user_id INT PRIMARY KEY,
user_name VARCHAR(100),
user_email VARCHAR(100)
);

CREATE TABLE movie(
movie_id INT PRIMARY KEY,
```

```
movie_name VARCHAR(100)
);

CREATE TABLE actors(
actor_id INT PRIMARY KEY,
actor_name VARCHAR(100),
actor_gender VARCHAR(100),
actor_type VARCHAR(100)
);

CREATE TABLE movie_type(
movie_id INT PRIMARY KEY,
movie_type VARCHAR(100),
movie_language VARCHAR(100),
movie_time VARCHAR(100)
);

CREATE TABLE theater(
theater_id INT PRIMARY KEY,
theater_name VARCHAR(100),
theater_location VARCHAR(100)
);

CREATE TABLE seat(
seat_id INT PRIMARY KEY,
seat_name VARCHAR(100)
);

CREATE TABLE show(
show_id INT PRIMARY KEY,
theater_id INT,
movie_id INT,
show_name VARCHAR(100),
show_time VARCHAR(100),
FOREIGN KEY(movie_id) REFERENCES movie(movie_id),
FOREIGN KEY(theater_id) REFERENCES theater(theater_id)
);

CREATE TABLE bookings(
booking_id INT PRIMARY KEY,
user_id INT,
show_id INT,
total_price VARCHAR(100),
payment_status VARCHAR(100),
FOREIGN KEY(user_id) REFERENCES user(user_id),
FOREIGN KEY(show_id) REFERENCES show(show_id)
);
```

```
CREATE TABLE booking_seat(
booking_seat_id INT PRIMARY KEY,
booking_id INT,
seat_id INT,
FOREIGN KEY(booking_id) REFERENCES bookings(booking_id),
FOREIGN KEY(seat_id) REFERENCES seat(seat_id)
);
```

Explanation:

1NF – seat numbers had multiple values, so a separate seat table is created.

2NF – partial dependency removed by separating user, movie and theatre into different tables.

3NF – transitive dependency removed by not storing everything in one table. Movie details and seat pricing are stored separately.

A3. Assumptions

1. Each user is uniquely identified using user_id.

2. One booking belongs to only one user.

3. A show represents a specific movie at a specific time.

4. A movie can have multiple shows in different theatres.

5. Each theatre has a fixed location.

6. Each booking can include multiple seats, so a separate booking_seat table is used.

7. Each seat is uniquely identified by seat_id.

8. Payment status is for each booking, not for each seat.

9. Total price is calculated based on number of seats and pricing.

10. User email is unique.

SECTION B – DATABASE DESIGN FROM REQUIREMENTS

Some important questions were considered before designing:

MOVIE

1. Can a movie have multiple languages and genres
2. Should language and genre be separate or simple attributes
3. Can same movie run in multiple theatres

THEATRE

1. Can a theatre have multiple screens
2. Does each screen have fixed seats

SHOW

1. Can same screen run multiple shows
2. Can shows overlap
3. Is pricing fixed or dynamic

SEAT

1. Is availability tracked per show or screen
2. Can seat types have different prices

BOOKING

1. Can user book multiple seats
2. Can booking be cancelled

USER

1. Can user have multiple bookings
2. Is email unique

LOCATION

1. Does theatre belong to one city

B2. Database Design

USERS
user_id is primary key, stores name and email

MOVIE
movie_id is primary key, stores movie name

MOVIE_DETAILS
stores language and genre

CITY
stores city_id and city_name

THEATRE
linked to city using foreign key

SCREEN
linked to theatre

SEAT_TYPE
stores type of seat

SEAT
linked to screen and seat type

SHOW
linked to movie and screen

SHOW_PRICING
stores price based on seat type

BOOKING
stores booking details

BOOKING_SEAT
links booking and seat

SHOW_SEAT
tracks availability

B3. Normalization

Booking table is modified by removing user_id.

New tables are created:

BOOKING_USER → links booking and user
BOOKING_USER_SEAT → links booking, user and seat

This removes redundancy and improves structure.

SECTION C – QUERIES AND TRANSACTIONS

C1. Retrieve bookings of a user in date range

```sql
SELECT b.booking_id, m.movie_name, t.theatre_name, s.show_time, se.seat_number
FROM bookings b
JOIN users u ON b.user_id = u.user_id
JOIN shows s ON b.show_id = s.show_id
JOIN movies m ON s.movie_id = m.movie_id
JOIN screen sc ON s.screen_id = sc.screen_id
JOIN theatre t ON sc.theatre_id = t.theatre_id
JOIN booking_seat bs ON b.booking_id = bs.booking_id
JOIN seat se ON bs.seat_id = se.seat_id
WHERE u.user_id = 1
AND b.booking_date BETWEEN '2026-01-01' AND '2026-12-31';
```

C2. Most frequently booked movie

```sql
SELECT m.movie_name, COUNT(b.booking_id) AS total_bookings
FROM bookings b
JOIN shows s ON b.show_id = s.show_id
JOIN movie m ON s.movie_id = m.movie_id
GROUP BY m.movie_name
ORDER BY total_bookings DESC
LIMIT 1;
```

C3. Shows with booked and available seats

```sql
SELECT s.show_id, m.movie_name, s.show_time
FROM shows s
JOIN movie m ON s.movie_id = m.movie_id
JOIN screen sc ON s.screen_id = sc.screen_id
JOIN theatre t ON sc.theatre_id = t.theatre_id
JOIN show_seat ss ON s.show_id = ss.show_id
WHERE t.theatre_id = 1;
```

C4. Transaction – Booking operation

```sql
SELECT seat_id
FROM show_seat
WHERE show_id = 101
AND seat_id IN (1,2,3)
AND status = 'AVAILABLE';

INSERT INTO bookings(booking_id, user_id, show_id, booking_date, total_price,
payment_status)
VALUES (1001,1,101,NOW(),500,'Paid');

INSERT INTO booking_seat(booking_id, seat_id)
VALUES (1001,1),(1001,2),(1001,3);

UPDATE show_seat
SET status = 'Booked'
WHERE show_id = 101
AND seat_id IN (1,2,3);

COMMIT;
```

This structure removes redundancy, avoids anomalies and follows normalization rules up to 3NF.